Sistemas Operacionais

Introdução a processo e compreensão de PCB (Bloco de controle de processos)

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Objetivos da Aula de Hoje

- **Compreender** o conceito de processo, diferenciando-o de programa e compreendendo seu papel como unidade de execução gerenciada pelo sistema operacional.
- **Analisar** o ciclo de vida dos processos, identificando seus estados (novo, pronto, executando, bloqueado) e as transições entre eles.
- **Entender** a estrutura e a função do Bloco de Controle de Processo (PCB), reconhecendo as principais informações armazenadas e sua importância para o gerenciamento pelo kernel.
- **Aplicar** conceitos de gerenciamento de processos, incluindo criação, término e troca de contexto, por meio da análise de diagramas e preenchimento prático de um PCB.

Para pensar:

Se um programa é apenas um conjunto de instruções…

o que realmente está acontecendo quando ele “ganha vida” como processo?

- Por que o sistema operacional precisa controlar cada processo?
- O que aconteceria se todos os processos executassem simultaneamente, sem controle?
- Um processo parado ainda “existe”? Em que sentido?
- Quem realmente está no controle: o programa ou o sistema operacional?

Programa vs Processo: O Estático e o Dinâmico

Programa

- Arquivo passivo armazenado no disco rígido.
- Contém uma lista estática de instruções e dados.
- Não consome CPU ou memória RAM enquanto está no disco.

A Receita

Um conjunto de instruções escritas em um papel.

Processo

- Entidade ativa carregada na memória principal.
- Instância de um programa em execução com contexto próprio.
- Possui contador de programa, registradores e pilha.

O Cozinheiro

A ação de seguir a receita e de preparar o prato.

O que é um processo?

Um processo é um programa em execução, incluindo seu estado atual.

Processo =
- Código (instruções)
- Dados
- Contexto de execução

Componentes de um processo
- Contador de programa (PC)
- Registradores da CPU
- Pilha (stack)
- Área de dados
- Estado atual (executando, pronto, bloqueado)

Exemplo de execução de 4 processos em multiprogramação.

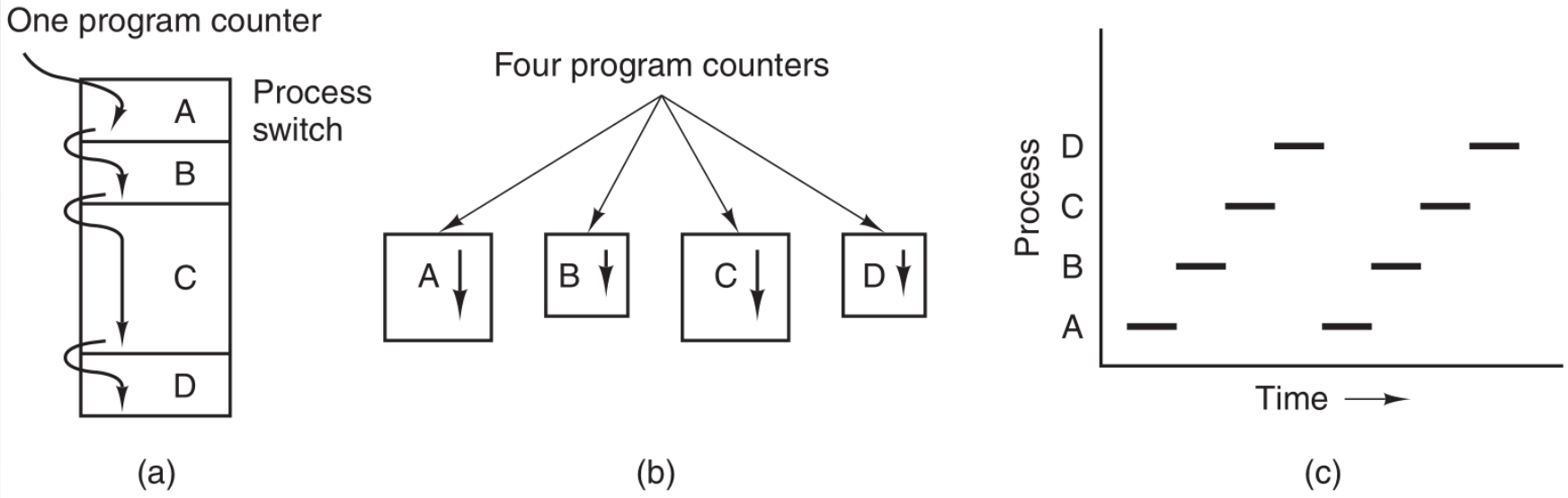


Figure 2-1. (a) Multiprogramming four programs. (b) Conceptual model of four independent, sequential processes. (c) Only one program is active at once.

https://blog.csdn.net/Hongwei_1990

O Papel do Sistema Operacional como Gerenciador

A Transformação

O SO é o agente ativo que retira o código inerte do disco e o insufla com vida na memória RAM, criando o contexto necessário para a execução.

Programa → Processo

01 **Alocação de Recursos**
Distribui fatias de tempo de CPU e segmentos de memória para cada processo.

02 **Escalonamento**
Decide a ordem e a prioridade de execução para maximizar a eficiência do sistema.

03 **Abstração de Hardware**
Fornece uma interface padronizada para acesso a periféricos e arquivos.

Classificação de Processos

Os processos são geralmente categorizados com base em qual recurso eles utilizam de forma mais intensiva:

1. I/O Bound (Intensivos em E/S)
   - Estes processos passam a maior parte do seu ciclo de vida esperando por operações de Entrada e Saída (como leitura de disco, teclado ou rede).
   - **Característica**: Possuem muitas "rajadas" (bursts) de processamento, porém elas são muito curtas.
   - **Exemplo**: Um editor de texto ou um navegador web, que ficam muito tempo esperando o usuário digitar ou os dados chegarem da internet.
2. CPU Bound (Intensivos em Processamento)
   - Estes processos exigem muito poder de cálculo e passam a maior parte do tempo utilizando o processador.
   - **Característica**: Possuem períodos longos de uso de CPU, com pouca espera por periféricos.
   - **Exemplo**: Softwares de renderização de vídeo, cálculos matemáticos complexos ou mineração de dados.

Hierarquia de processos

- **UNIX**: Adota uma estrutura hierárquica onde um processo "pai" cria um processo "filho", formando grupos de processos.
- **Windows**: Não utiliza hierarquia; todos os processos são criados iguais, sem uma relação formal de pai e filho.
- **Handle**: O Windows utiliza handles para gerenciar o acesso aos processos.

Inicialização de processos

Principais Eventos para Criação de Processos

- Início do sistema: Quando o computador é ligado (boot), o sistema operacional cria diversos processos. Alguns são processos de primeiro plano (que interagem com o usuário) e outros são processos de segundo plano (conhecidos como daemons no Linux ou serviços no Windows), que cuidam de funções como gerenciar a rede, impressão ou atualizações.
- Execução de chamada ao sistema de criação de processos: Um processo que já está rodando pode solicitar a criação de outro para ajudá-lo em uma tarefa.
    - Exemplo: No Unix/Linux, usa-se a chamada fork(). No Windows, a função CreateProcess(). Isso permite que um programa se divida em partes menores para processar dados de forma mais eficiente.

Inicialização de um processo

Principais Eventos para Criação de Processos

- Solicitação do usuário para criar um novo processo: Este é o evento mais comum no dia a dia. Acontece quando você:
  - Clica duas vezes no ícone de um navegador.
  - Digita um comando no terminal (como ls ou dir).
  - Abre um aplicativo de música.
- Início de um job em lote (batch job):Comum em sistemas de grande porte (Mainframes) ou servidores de processamento de dados. O usuário submete uma lista de tarefas ("lote") para o sistema. Quando o sistema tem recursos disponíveis, ele lê o próximo trabalho da fila e cria um novo processo para executá-lo sem interrupção humana.

Finalização de processo

Término normal (voluntário):

O processo finaliza sua execução de forma planejada, normalmente ao concluir sua tarefa, utilizando chamadas como exit().

Término por erro (voluntário ou involuntário):

Ocorre quando o próprio processo detecta uma condição de erro (ex: falha de validação) e decide encerrar sua execução.

Finalização de processo

Erro fatal (involuntário):

O processo é encerrado pelo sistema operacional devido a falhas graves, como:

- violação de memória (segmentation fault)
- instrução ilegal
- exceções não tratadas

Término por outro processo (involuntário):

Um processo pode ser finalizado externamente, por exemplo:

- kill (Linux)
- TerminateProcess (Windows)

Finalização de processo – Chamadas de Sistema

Padrões:

exit() → término voluntário (processo atual)

kill() → envio de sinal para outro processo

TerminateProcess() → finalização forçada no Windows

Adicionais:

wait() / waitpid() → coleta do status (evita zumbis)

Finalização de processo – Erros comuns

O sistema operacional encerra o processo devido a falhas graves:

SIGSEGV (falha de segmentação) (Geralmente Signal 11)

- O programa tentou ler ou gravar em uma área da memória que não tem permissão para acessar.

SIGILL (instrução ilegal) (Geralmente Signal 4)

- O processador tentou executar uma instrução que ele não reconhece ou não consegue processar.

SIGABRT (abort) (Geralmente Signal 6)

- O programa solicitou ao sistema operacional que o encerre imediatamente, geralmente devido a um erro detectado pela própria aplicação ou pela biblioteca padrão (como libc).

Ciclo de Vida: Os Estados de um Processo

01 **Novo (New)**

O processo está sendo criado pelo sistema operacional e ainda não foi admitido na fila de prontos.

02 **Pronto (Ready)**

O processo está carregado na memória principal e aguarda a disponibilidade da CPU para iniciar sua execução.

03 **Executando (Running)**

As instruções do processo estão sendo efetivamente processadas pela CPU no momento atual.

04 **Bloqueado (Waiting)**

O processo aguarda a ocorrência de um evento externo, como a conclusão de uma operação de entrada ou saída.

05 **Finalizado (Terminated)**

O processo encerrou sua execução e o sistema operacional está liberando os recursos que ele ocupava.

Diagrama de Estados e Transições

01 **Admissão**
O processo passa de Novo para Pronto assim que o SO reserva os recursos iniciais.

02 **Despacho (Dispatch)**
O escalonador seleciona um processo Pronto para ocupar a CPU (Executando).

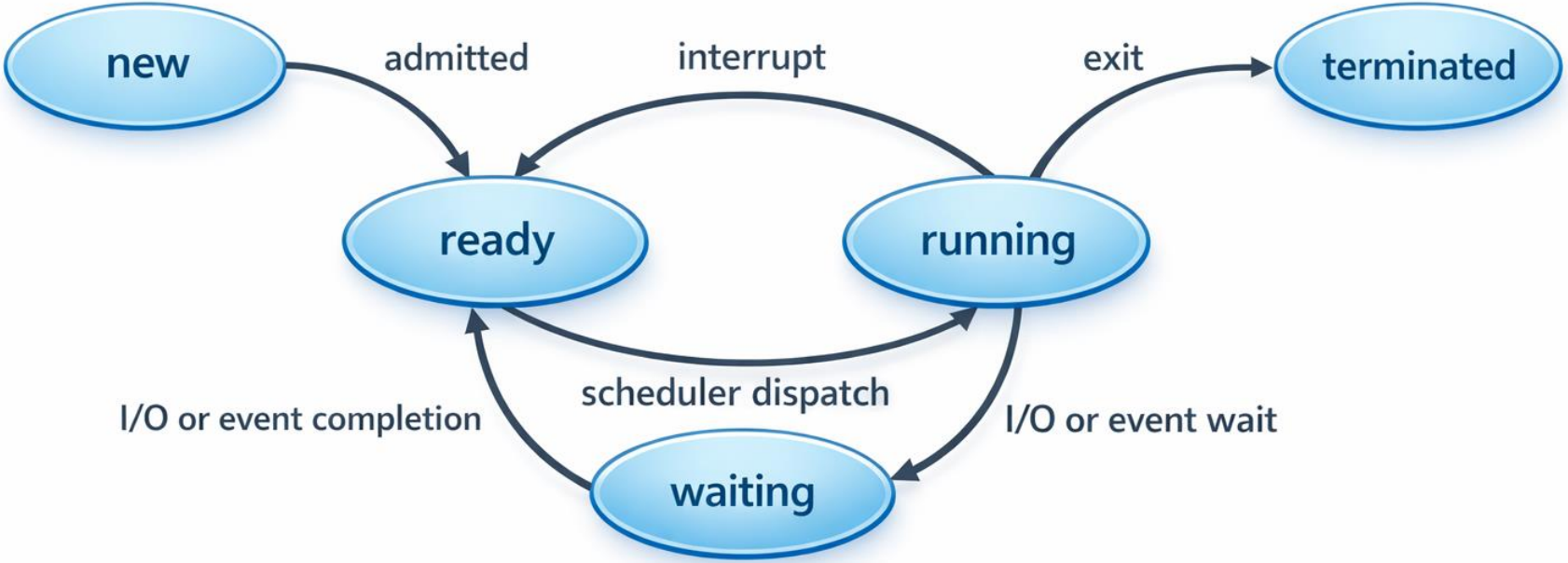
03 **Interrupção**
O processo em Execução volta para Pronto (ex: fim do quantum de tempo).

04 **Espera de I/O**
O processo vai de Executando para Bloqueado ao solicitar dados externos.

05 **Retorno ao Pronto**
Após o evento de I/O ser concluído, o processo volta para a fila de Pronto.

Fluxo Dinâmico: Novo → Pronto ↔ Executando → Bloqueado → Finalizado

Exemplo: Gerenciamento do CPU



PCB: O Bloco de Controle de Processo

PCB

Definição

O PCB é uma estrutura de dados fundamental no kernel que armazena todas as informações vitais sobre um processo específico.

A "Memória" do SO

Atua como o registro histórico do processo, permitindo que o SO saiba exatamente onde a execução parou e quais recursos estão em uso.

Multiprogramação

É a peça-chave que possibilita a alternância rápida entre múltiplos processos, criando a ilusão de execução simultânea.

Localização Segura

Os PCBs residem exclusivamente no espaço de memória do kernel, protegidos contra acessos ou modificações por processos de usuário.

Anatomia de um PCB: O que o SO Armazena?

01 **PID (Process Identifier)**
Número único que identifica o processo no sistema operacional.

02 **Contador de Programa (PC)**
Endereço da próxima instrução a ser executada pelo processo.

03 **Registradores da CPU**
Estado dos registradores para restauração durante a troca de contexto.

04 **Gerenciamento de Memória**
Tabelas de páginas, limites de memória e segmentos alocados.

SPACE KERNEL

Estrutura PCB

0x00: PID_DATA
0x08: PROCESS_STATE
0x10: PROGRAM_COUNTER
0x18: CPU_REGISTERS
0x40: MEMORY_LIMITS
0x50: OPEN_FILES_LIST
0x60: SCHEDULING_INFO

Operações Fundamentais: Criação e Término

01

Criação (fork)

O processo pai cria um processo filho que é uma cópia quase idêntica de si mesmo, herdando recursos e contexto.

02

Execução (exec)

Frequentemente usada após o fork para substituir o espaço de endereçamento do processo por um novo programa.

03

Término (exit)

O processo solicita ao SO sua própria destruição, liberando recursos e devolvendo um código de status ao pai.

"Processos formam uma árvore hierárquica, onde cada filho tem um pai responsável por seu ciclo de vida."

Troca de Contexto (Context Switch)

01 **Salvar Estado**
O SO salva o estado atual da CPU (registradores, PC) no PCB do processo que está saindo.

02 **Carregar Estado**
O SO recupera o estado salvo no PCB do próximo processo a ser executado.

03 **Retomar Execução**
A CPU volta a processar as instruções a partir do ponto exato onde o novo processo parou.

O Custo da Multitarefa

A troca de contexto é puramente **overhead** administrativo. Durante esse tempo, a CPU não executa nenhum trabalho útil para o usuário.

Sistemas modernos realizam milhares de trocas por segundo, criando a **ilusão de simultaneidade** perfeita para o usuário final.

Gatilhos comuns: Interrupções de hardware, chamadas de sistema ou expiração do tempo de CPU (quantum).

Processos em Primeiro Plano vs Segundo Plano

Processo em Primeiro Plano (Foreground)

- Interage diretamente com o usuário
- Executado no terminal ativo
- Recebe entrada (teclado) e envia saída (tela)
- Bloqueia o terminal até finalizar

```bash
./programa
```

Processos em Primeiro Plano vs Segundo Plano

Processo em Segundo Plano (Background)

- Executa sem interação direta
- Não bloqueia o terminal
- Pode rodar por longos períodos
- Ideal para tarefas assíncronas

```bash
./programa &
```

Controle de Processos (Linux/WSL)

- jobs → lista processos em background
- fg → traz para primeiro plano
- bg → envia para segundo plano

Exemplo:

fg %1

bg %1

Processos Zumbies

Criando um Zumbis

- Um processo que já terminou… mas ainda não “morreu” completamente.

Como isso acontece:

- Processo que **já finalizou sua execução (exit)**
- Ainda mantém uma entrada na tabela de processos (**PCB parcialmente ativo**)
- Fica aguardando o processo pai ler seu status (**wait()**)

Demonstração Prática

01 Ambiente Linux / WSL

```
$ ps aux
```

Lista todos os processos do sistema, exibindo o usuário (USER), o identificador (PID) e o consumo de CPU/Memória.

Monitoramento em Tempo Real

Use top ou htop no Linux para visualizar as transições de estado e o escalonamento dinâmico.

02 Ambiente Windows

```
C:\> tasklist
```

Exibe uma lista de processos ativos, o nome da imagem e o uso de memória no prompt de comando ou PowerShell.

A Raiz do Sistema

Observe o PID 1 ou 4 (init ou systemd), o processo ancestral de todos os outros processos no sistema operacional.

Exemplo Mobile - Android

01 Ambiente Android

```
$ adb ps aux
```

Lista todos os processos do sistema, exibindo o usuário (USER), o identificador (PID) e o consumo de CPU/Memória.

Monitoramento em Tempo Real

Use top no Android para visualizar as transições de estado e o escalonamento dinâmico.

```
adb shell top
```

Observe o PID 1 (**init**) no Android, responsável por iniciar o sistema. Em seguida, identifique o processo **zygote**, que é o verdadeiro responsável por criar a maioria dos processos de aplicações. Compare essa arquitetura com o Linux tradicional, onde o PID 1 possui um papel mais central como ancestral direto.

Exemplo Android – A Prática

- O Zygote pode imediatamente criar processos chamados **unspecialized app processes (USAP)** ou aguardar para criar processos conforme necessário pelas aplicações. A primeira opção deve ser habilitada por meio de uma propriedade do sistema ou por um comando do Android Debug Bridge. Para mais informações sobre como configurar o Zygote para criar processos, consulte a documentação: **Enable unspecialized app processes pool**.

FONTE: https://source.android.com/docs/core/runtime/zygote

Exemplo de Zumbi em C:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Erro no fork");
        return 1;
    }

    if (pid == 0) {
        // Processo filho
        printf("Filho: PID = %d, PPID = %d\n", getpid(), getppid());
        printf("Filho encerrando...\n");
        exit(0);
    } else {
        // Processo pai
        printf("Pai: PID = %d, filho criado com PID = %d\n", getpid(), pid);
        printf("Pai vai dormir por 30 segundos sem chamar wait().\n");
        printf("Durante esse tempo, o filho vira zumbi.\n");

        sleep(30);

        printf("Pai encerrando.\n");
    }

    return 0;
}
```

Exemplo de Zumbi resolvido em C:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Erro no fork");
        return 1;
    }

    if (pid == 0) {
        printf("Filho: PID = %ld, PPID = %ld\n", (long)getpid(), (long)getppid());
        printf("Filho encerrando...\n");
        exit(0);
    } else {
        int status;

        printf("Pai: PID = %ld, filho criado com PID = %ld\n",
               (long)getpid(), (long)pid);

        waitpid(pid, &status, 0);

        printf("Pai coletou o status do filho.\n");
        printf("Filho nao virou zumbi.\n");
    }

    return 0;
}
```

Exemplo de Zumbi resolvido em C:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {

    pid_t pid = fork();
    if (pid < 0) {
        perror("Erro no fork");
        return 1;
    }
```

```c
int main(void) {

    pid_t pid = fork();
    if (pid < 0) {
        perror("Erro no fork");
        return 1;
    }

    if (pid == 0) {
        printf("Filho: PID = %ld, PPID = %ld\n",
(long)getpid(), (long)getppid());
        printf("Filho encerrando...\n");
        exit(0);
    } else {
        int status;

        printf("Pai: PID = %ld, filho criado com PID =
%ld\n",
               (long)getpid(), (long)pid);
```

```
        printf("Filho encerrando...\n");
        exit(0);
    } else {
        int status;

        printf("Pai: PID = %ld, filho criado com PID =
%ld\n",
               (long)getpid(), (long)pid);

        waitpid(pid, &status, 0);

        printf("Pai coletou o status do filho.\n");
        printf("Filho nao virou zumbi.\n");
    }

    return 0;
}
```

Exemplo de Zumbi em C para Windows:

```c
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

int main() {
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    char cmd[256];

    // Primeiro cria o processo (temporariamente com comando simples)
    if (!CreateProcessA(
            NULL,
            "cmd /c echo Iniciando filho...",
            NULL,
            NULL,
            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi)) {
        printf("Erro ao criar processo. Codigo: %lu\n", GetLastError());
        return 1;
    }

    // Agora monta o comando com o PID correto
    sprintf(cmd,
        "cmd /c echo Filho: PID = %lu && echo Filho encerrando...",
        pi.dwProcessId
    );

    // Executa o comando formatado (novo processo só para exibir corretamente)
    system(cmd);

    printf("Pai: PID = %lu, filho criado com PID = %lu\n",
           GetCurrentProcessId(), pi.dwProcessId);

    WaitForSingleObject(pi.hProcess, INFINITE);

    printf("Pai coletou o status do filho.\n");
    printf("Filho nao virou zumbi.\n");

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
}
```

Resultado:

- Ao executar no Windows você deve observar seguinte saída:

```
PS C:\Users\PICHAU\Documents\CatólicaSC\S.O\Aula 04\CodigosC>  & 'c:\Users\PICHAU\.vscode\extensions\ms-vscode.cpptools-1.30.5-win32-x64\debugAdapters\bin\WindowsDebugLauncher.e
xe' '--stdin=Microsoft-MIEngine-In-bzejr5rj.xnb' '--stdout=Microsoft-MIEngine-Out-u4mf5dmi.xqh' '--stderr=Microsoft-MIEngine-Error-q1zgwtcc.hzj' '--pid=Microsoft-MIEngine-Pid-4h
ekjpws.0ur' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Iniciando filho...
Filho: PID = 21416
Filho encerrando...
Pai: PID = 5988, filho criado com PID = 21416
Pai coletou o status do filho.
Filho nao virou zumbi.
```

Sua vez, vamos testar na prática!

Objetivo:

- Testar o código em C para diferenciar programa de processos.
- Entender o que é um processo Zumbi.

Código inicial em Linux e WSL.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Erro no fork");
        return 1;
    }

    if (pid == 0) {
        printf("Filho: PID = %ld, PPID = %ld\n", (long)getpid(), (long)getppid());
        printf("Filho encerrando...\n");
        exit(0);
    } else {
        int status;

        printf("Pai: PID = %ld, filho criado com PID = %ld\n",
               (long)getpid(), (long)pid);

        waitpid(pid, &status, 0);

        printf("Pai coletou o status do filho.\n");
        printf("Filho nao virou zumbi.\n");
    }

    return 0;
}
```

Código inicial em Windows.

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

int main() {
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    char cmd[256];

    // Primeiro cria o processo (temporariamente com comando simples)
    if (!CreateProcessA(
            NULL,
            "cmd’/c echo Iniciando filho...",
            NULL,

            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi)) {
        printf("Erro ao criar processo. Codigo: %lu\n", GetLastError());
        return 1;
    }

    // Agora monta o comando com o PID correto
    sprintf(cmd,
        "cmd /c’echo Filho: PID = %lu && echo Filho encerrando...",
        pi.dwProcessId
    );

    // Executa o comando formatado (novo processo só para exibir corretamente)
    system(cmd);

    printf("Pai: PID = %lu, filho criado com PID = %lu\n",
           GetCurrentProcessId(), pi.dwProcessId);

    WaitForSingleObject(pi.hProcess, INFINITE);

    printf("Pai coletou o status do filho.\n");
    printf("Filho nao virou zumbi.\n");

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
    }
```

NULL,

Código inicial em Windows.

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

int main() {
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    char cmd[256];

    // Primeiro cria o processo (temporariamente com
```

```
    char cmd[256];

    // Primeiro cria o processo (temporariamente com
comando simples)
    if (!CreateProcessA(
            NULL,
            "cmd /c echo Iniciando filho...",
            NULL,
                                                NULL,
            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi)) {
        printf("Erro ao criar processo. Codigo:
%lu\n", GetLastError());
        return 1;
    }

    // Agora monta o comando com o PID correto
    sprintf(cmd,
        "cmd /c echo Filho: PID = %lu && echo Filho
```

```
        pi.dwProcessId
    );

    // Executa o comando formatado (novo processo só
para exibir corretamente)
    system(cmd);

    printf("Pai: PID = %lu, filho criado com PID =
%lu\n",
           GetCurrentProcessId(), pi.dwProcessId);

    WaitForSingleObject(pi.hProcess, INFINITE);

    printf("Pai coletou o status do filho.\n");
    printf("Filho nao virou zumbi.\n");


    return 0;
}
```

Código inicial em Windows.

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

int main() {
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    char cmd[256];

    // Primeiro cria o processo (temporariamente com
```

```
    // Primeiro cria o processo (temporariamente com
comando simples)
    if (!CreateProcessA(
            NULL,
            "cmd /c echo Iniciando filho...",
            NULL,
                                                NULL,
            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi)) {
        printf("Erro ao criar processo. Codigo:
%lu\n", GetLastError());
        return 1;
    }

    // Agora monta o comando com o PID correto
    sprintf(cmd,
        "cmd /c echo Filho: PID = %lu && echo Filho
```

```
            &pi)) {
        printf("Erro ao criar processo. Codigo:
%lu\n", GetLastError());
        return 1;
    }

    // Agora monta o comando com o PID correto
    sprintf(cmd,
        "cmd /c echo Filho: PID = %lu && echo Filho
encerrando...",
        pi.dwProcessId
    );

    // Executa o comando formatado (novo processo só
para exibir corretamente)
    system(cmd);

    printf("Pai: PID = %lu, filho criado com PID =
%lu\n",
           GetCurrentProcessId(), pi.dwProcessId);

    WaitForSingleObject(pi.hProcess, INFINITE);
```

```
encerrando...",
        pi.dwProcessId
    );

    // Executa o comando formatado (novo processo só
para exibir corretamente)
    system(cmd);

    printf("Pai: PID = %lu, filho criado com PID =
%lu\n",
           GetCurrentProcessId(), pi.dwProcessId);

    WaitForSingleObject(pi.hProcess, INFINITE);

    printf("Pai coletou o status do filho.\n");
    printf("Filho nao virou zumbi.\n");

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
}
```

Observe a saída

- Execute os comandos “top” e “tasklist” e observe a saída.

No Windows:

```
tasklist /v
```

Not responding

No Linux:

```
top
```

Tasks: X total, X running, X sleeping, X stopped, 1 zombie

Hora de perguntar, dúvidas

Próxima aula

Introdução Threads

- Conceito, diferenças entre processos e threads e impacto no desempenho. Modelos de Threads: threads em espaço de usuário e em nível de kernel.
- Concorrência: execução em paralelo e compartilhamento de recursos. Problemas de Concorrência: regiões críticas e necessidade de sincronização. Exercício prático: dinâmica em grupo, simulando acesso concorrente a um recurso compartilhado, para identificar condições de corrida (race condition).